

OS Lecture-1

Introduction to Operating Systems

Dr. Javed Imran

Asst. Professor, CSED

Course Content

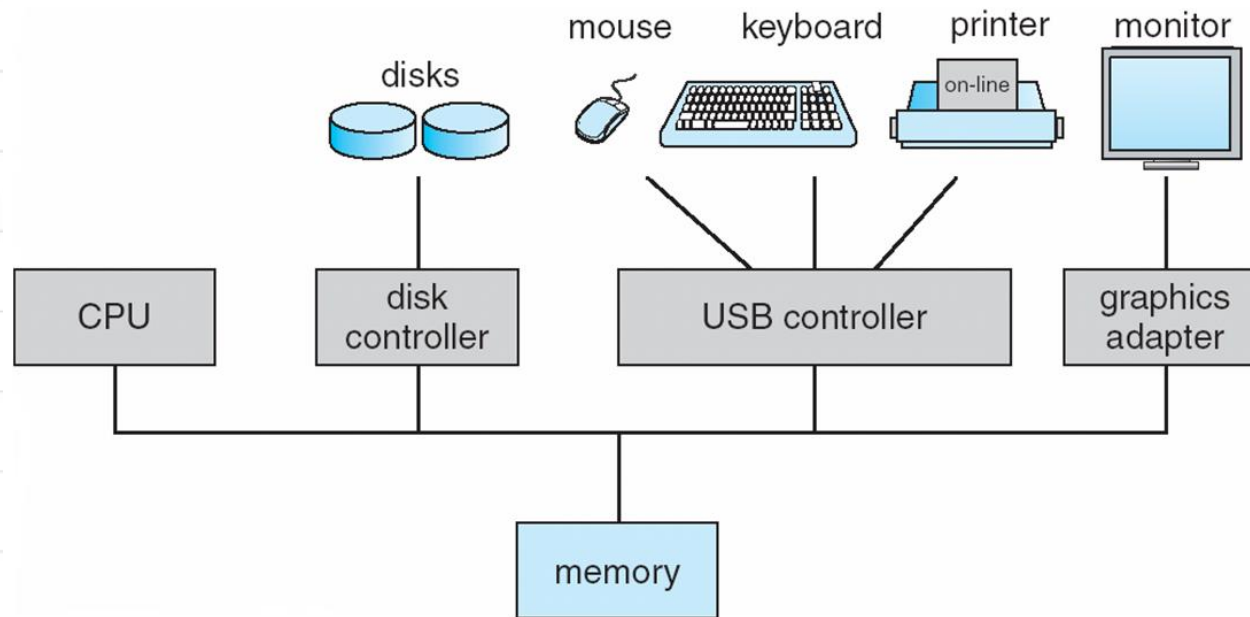
1. Introduction and System Structures
2. Process Management
3. Concurrency
4. Deadlock
5. Memory Management
6. File Systems
7. Disk Management
8. Protection and Security

Text/Reference Books

1. “Operating System Concepts” by Avi Silberschatz and Peter Galvin
2. “Operating Systems: Internals and Design Principles” by William Stallings
3. “Operating Systems: A Concept-Based Approach” by D M Dhamdhere

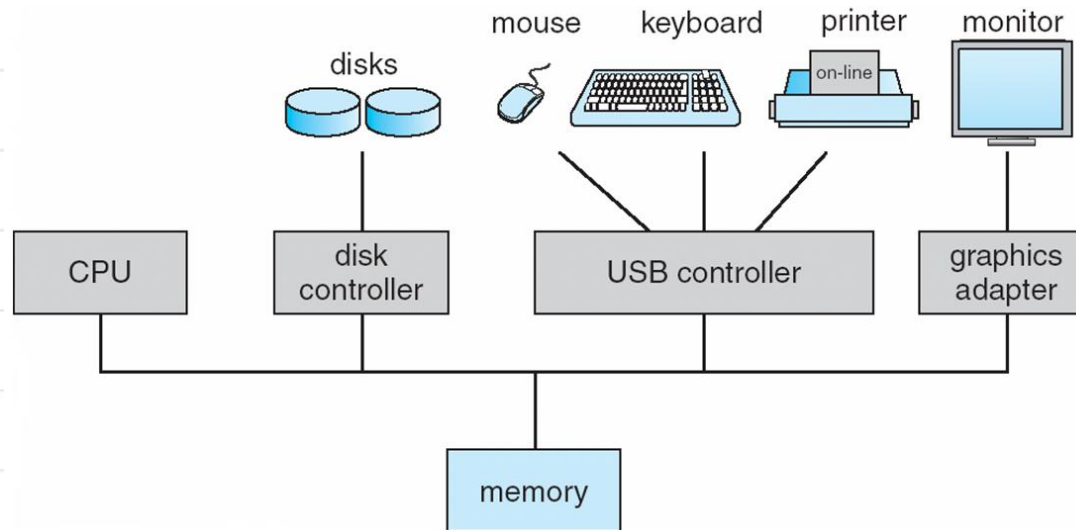
Computer System Organization and Architecture

Computer System Organization



- One or more CPUs, device controllers connect through common bus providing access to shared memory.
- Concurrent execution of CPUs and devices competing for memory cycles.

Computer-System Operation



- I/O devices and the CPU can execute concurrently.
- Each device controller is in charge of a particular device type.
- Each device controller has a local buffer.
- CPU moves data from/to main memory to/from local buffers.
- I/O is from the device to local buffer of controller.
- Device controller informs CPU that it has finished its operation by causing an **interrupt**.

Recap

1. **Which of the following components is *not* one of the four main components of a computer system?**
 - a) Hardware
 - b) Operating System
 - c) Compiler
 - d) Application Programs

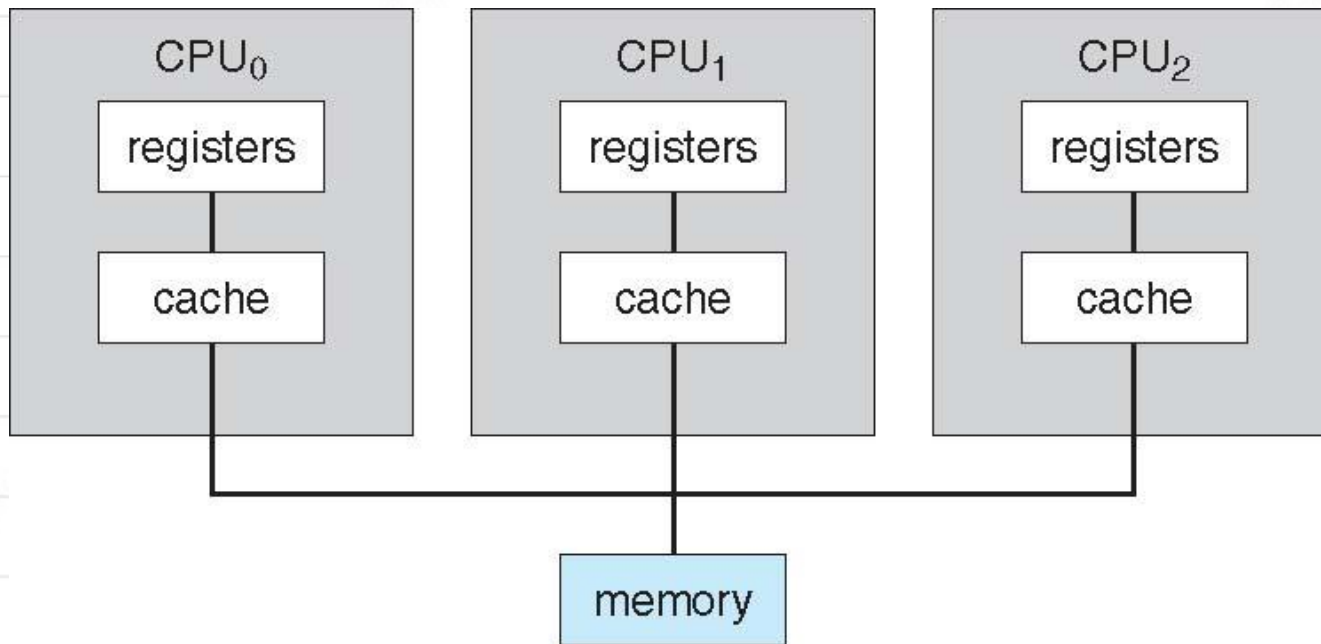
2. **True or False:** CPUs and I/O devices can execute concurrently in a typical computer system.

3. **What is the role of a device controller in computer-system operation?**
 - a) Acts as the primary CPU
 - b) Connects RAM to the network
 - c) Controls a specific device and communicates with the CPU
 - d) Serves as the power supply regulator

Computer System Architecture

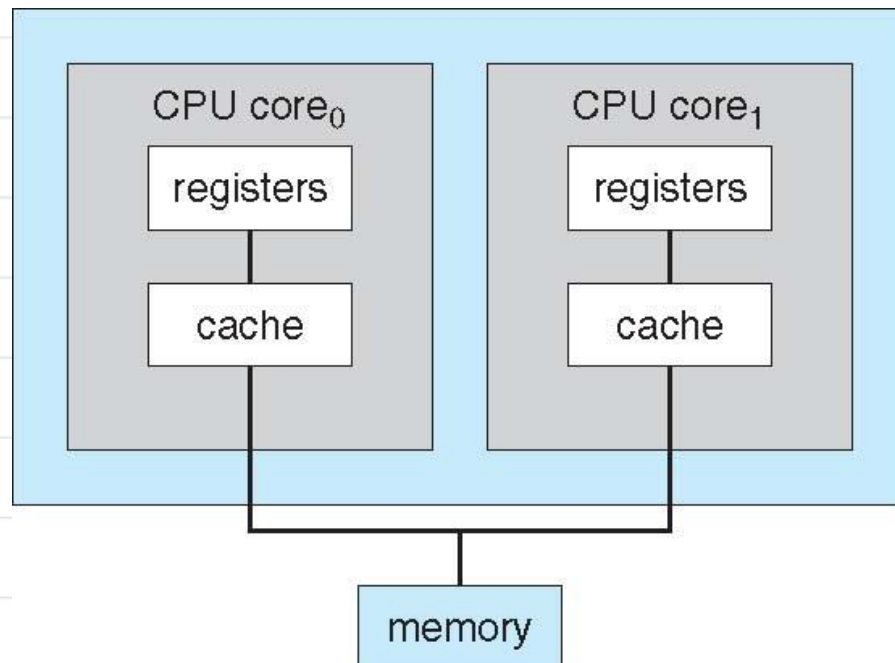
- Most systems use a single general-purpose processor, while some systems have special-purpose processors as well.
- **Multiprocessors** systems growing in use and importance.
 - Also known as **parallel systems, tightly-coupled systems**.
 - Advantages include:
 1. **Increased throughput**
 2. **Economy of scale**
 3. **Increased reliability** – graceful degradation or fault tolerance.
 - Two types:
 1. **Asymmetric Multiprocessing** – each processor is assigned a specific task. E.g. ARM Cortex-A73 (big) + Cortex-A53 (LITTLE)
 2. **Symmetric Multiprocessing** – each processor performs all tasks. E.g. Intel Core, AMD Ryzen

Symmetric Multiprocessing Architecture



A Dual-Core Design

- Multi-chip and **multicore**.
- Systems containing all chips.
 - Chassis containing multiple separate systems

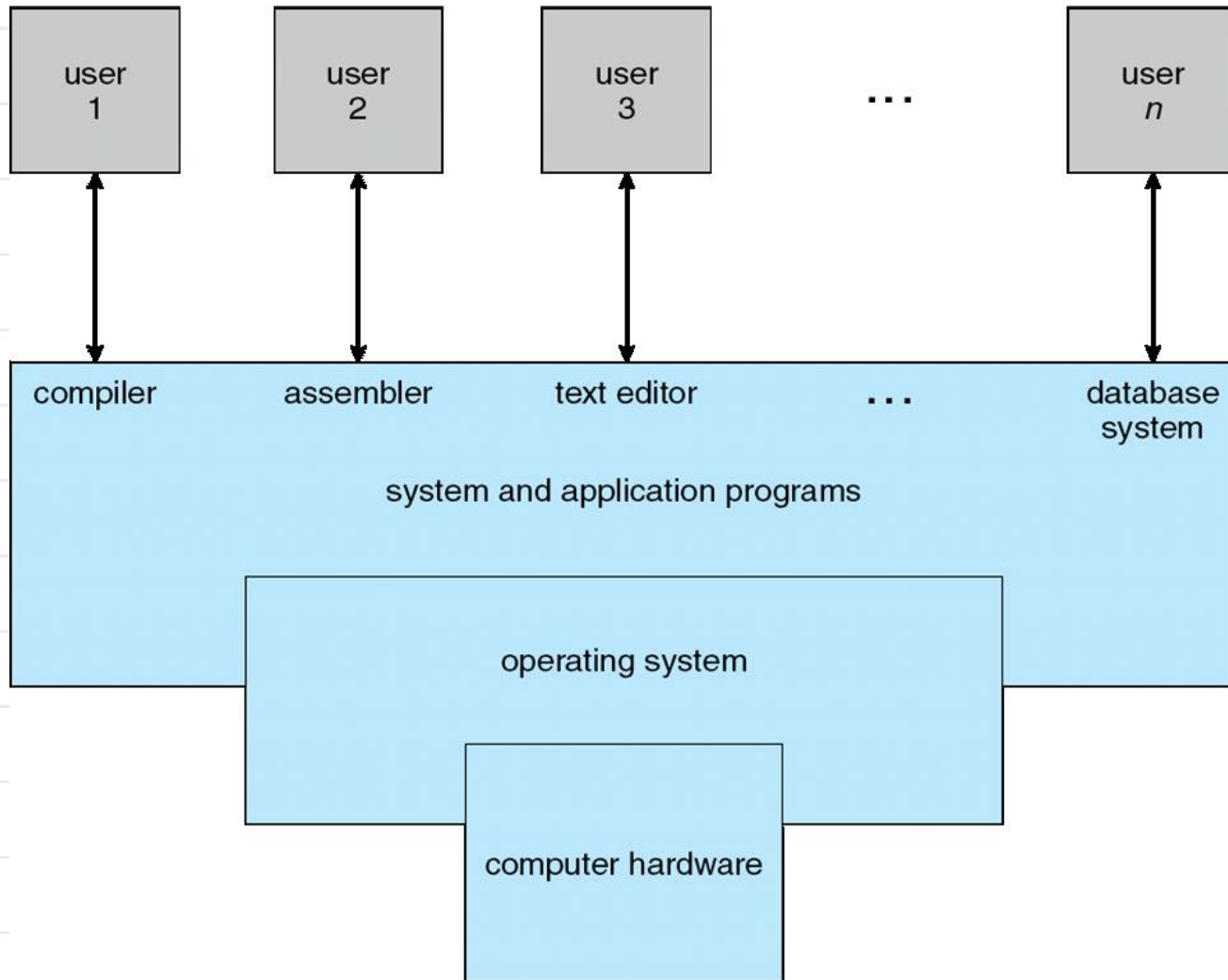


Recap

1. **Which of the following is a characteristic of Symmetric Multiprocessing (SMP)?**
 - a) Each processor handles a unique task only
 - b) All processors share memory and perform all tasks
 - c) Only one processor is active at a time
 - d) There is no need for a shared memory system

2. **Which of the following is an example of a special-purpose processor system?**
 - a) AMD Ryzen
 - b) Intel Core i7
 - c) ARM Cortex-A73 + A53 (big.LITTLE)
 - d) Apple M1

Four Components of a Computer System



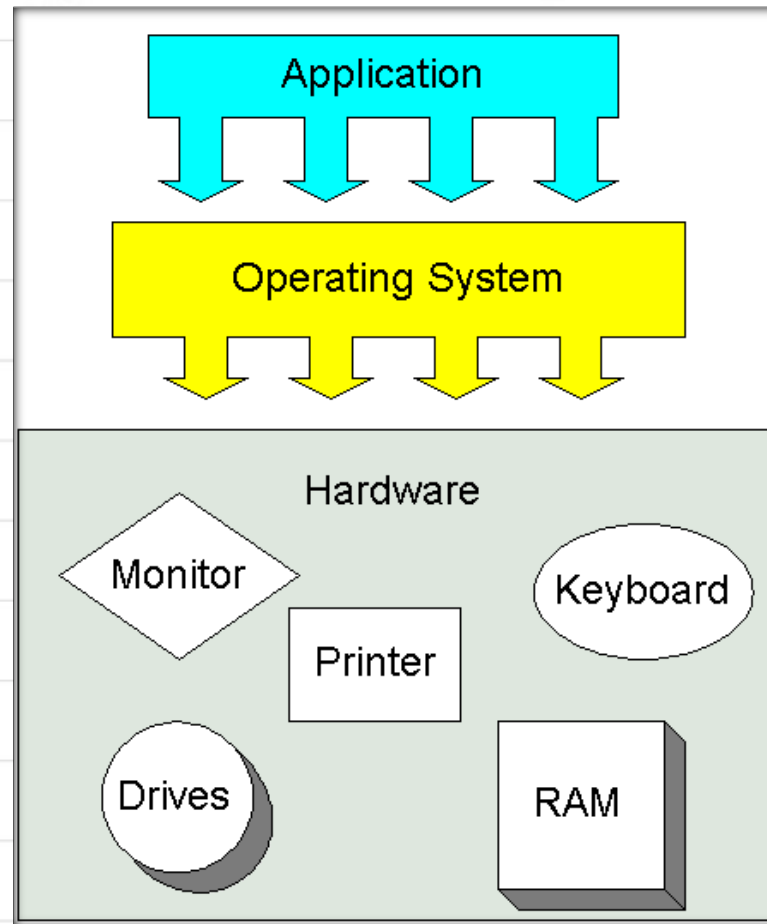
Computer System Structure

- Computer system can be divided into four components:
 - **Hardware** – provides basic computing resources
 - CPU, memory, I/O devices
 - **Operating system**
 - Controls and coordinates use of hardware among various applications and users
 - **Application programs** – define the ways in which the system resources are used to solve the computing problems of the users
 - Word processors, compilers, web browsers, database systems, video games
 - **Users**
 - People, machines, other computers

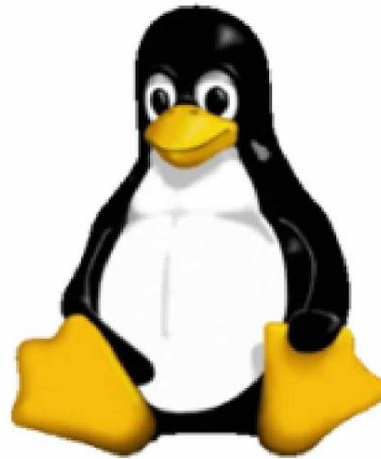
Operating System

What is an Operating System (OS)?

- An operating system (OS) is a collection of software that manages computer hardware and software resources and provides common services for computer programs.



Examples



Functions of OS

- **Processor Management:**

- In multiprogramming environment, OS decides which process gets the processor when and how much time.
- This function is called process scheduling.

- **Memory Management:**

- Refers to management of primary memory or main memory.
- Main memory is a large array of words or bytes where each word or byte has its own address.

- **Device Management:**

- OS manages device communication via their respective drivers.

- **File Management:**

- A file system is normally organized into directories for easy navigation and usage.
- These directories may contain files and other directions.

Other Important Activities

- **Security:**
 - By means of password and similar other techniques, preventing unauthorized access to programs and data.
- **Control over system performance:**
 - Recording delays between request for a service and response from the system.
- **Job accounting:**
 - Keeping track of time and resources used by various jobs and users.
- **Error detecting aids:**
 - Production of dumps, traces, error messages and other debugging and error detecting aids.
- **Coordination between other softwares and users:**
 - Coordination and assignment of compilers, interpreters, assemblers and other software to the various users of the computer systems.

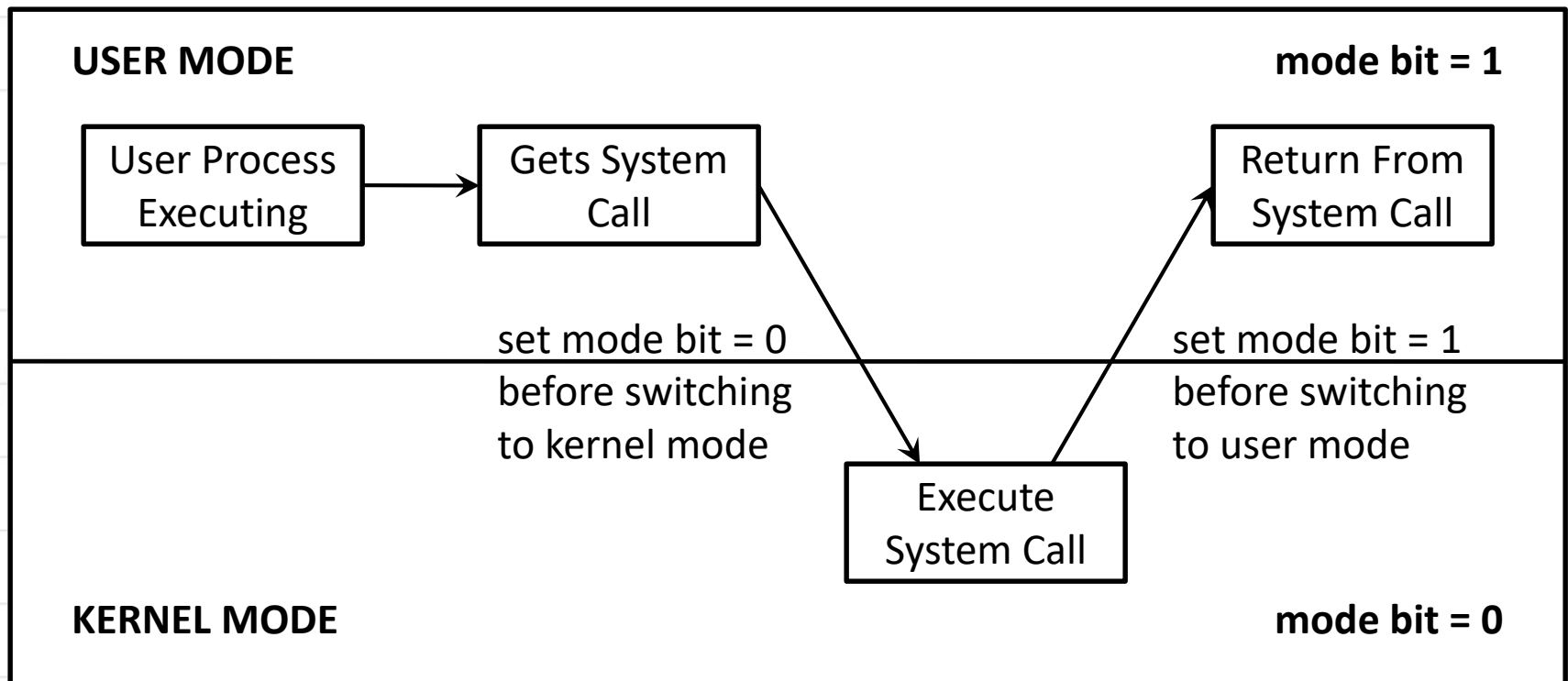
Operating System Operations

Operating System Operations

- **Interrupt driven** (hardware and software).
 - Hardware interrupt by one of the devices.
 - Software interrupt (**exception** or **trap**):
 - Software error (e.g., division by zero)
 - Request for operating system service
 - Other process problems include infinite loop, processes modifying each other or the operating system

Operating System Operations (cont.)

- **Dual-mode** operation allows OS to protect itself and other system components:
 1. **User mode**: OS is running a user application
 2. **Kernel mode**: OS executing some privileged instructions



Dual-modes of Operating System

Aspect	User Mode	Kernel Mode
Privilege Level	Lower-privileged	Higher-privileged
Access to Hardware	Restricted	Unrestricted
Access to System Memory	Limited	Full access
Execution Environment	User-level applications	Operating System and Kernel components
Error Isolation	Processes in User Mode are isolated	Kernel manages process isolation
Purpose	Run user applications	Manage system resources and hardware
Exception Handling	Limited exception handling capabilities	Comprehensive exception handling
Stability	Application crashes do not crash OS	Kernel issues can crash the entire OS

Recap

1. **What is 'process scheduling'?**
 - a) Keeping track of user logins
 - b) Assigning memory addresses
 - c) Deciding which process runs next
 - d) Managing file folders
2. **Match the mode with the description:** Kernel Mode, User Mode
 - a) Executes application programs
 - b) Executes privileged instructions
3. **What does a software interrupt typically result from?**
 - a) Power outage
 - b) Division by zero
 - c) Hardware failure
 - d) Manual reset
4. **True or False:** Crashes in User Mode applications can cause the entire operating system to fail.

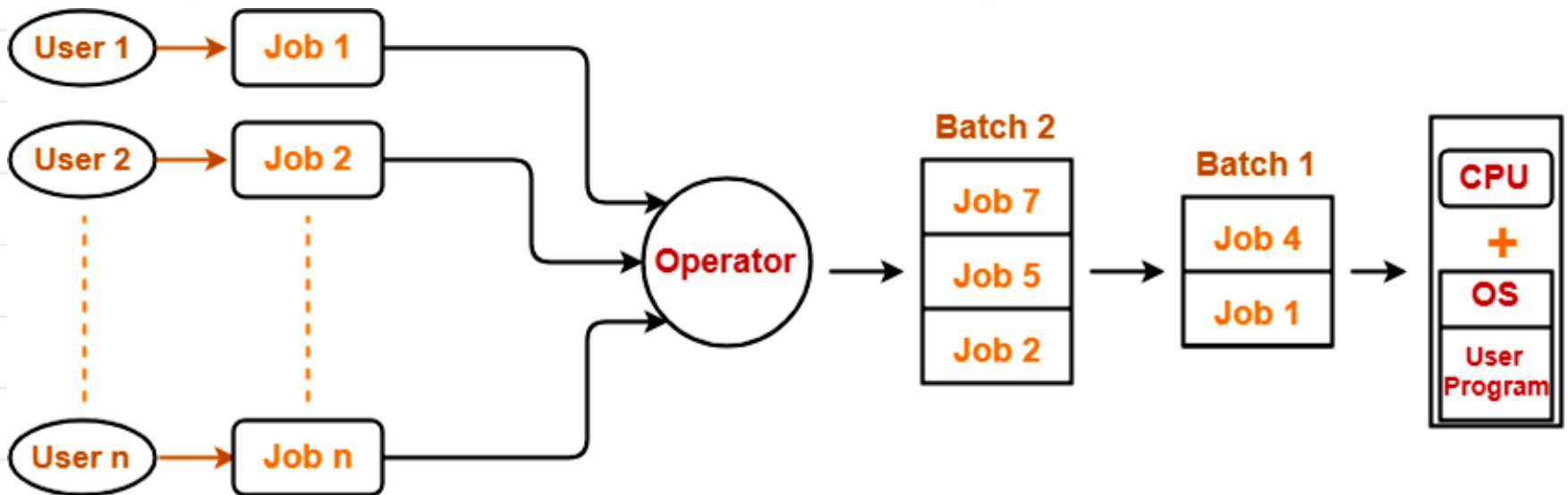
Types of Operating Systems

Types of Operating System

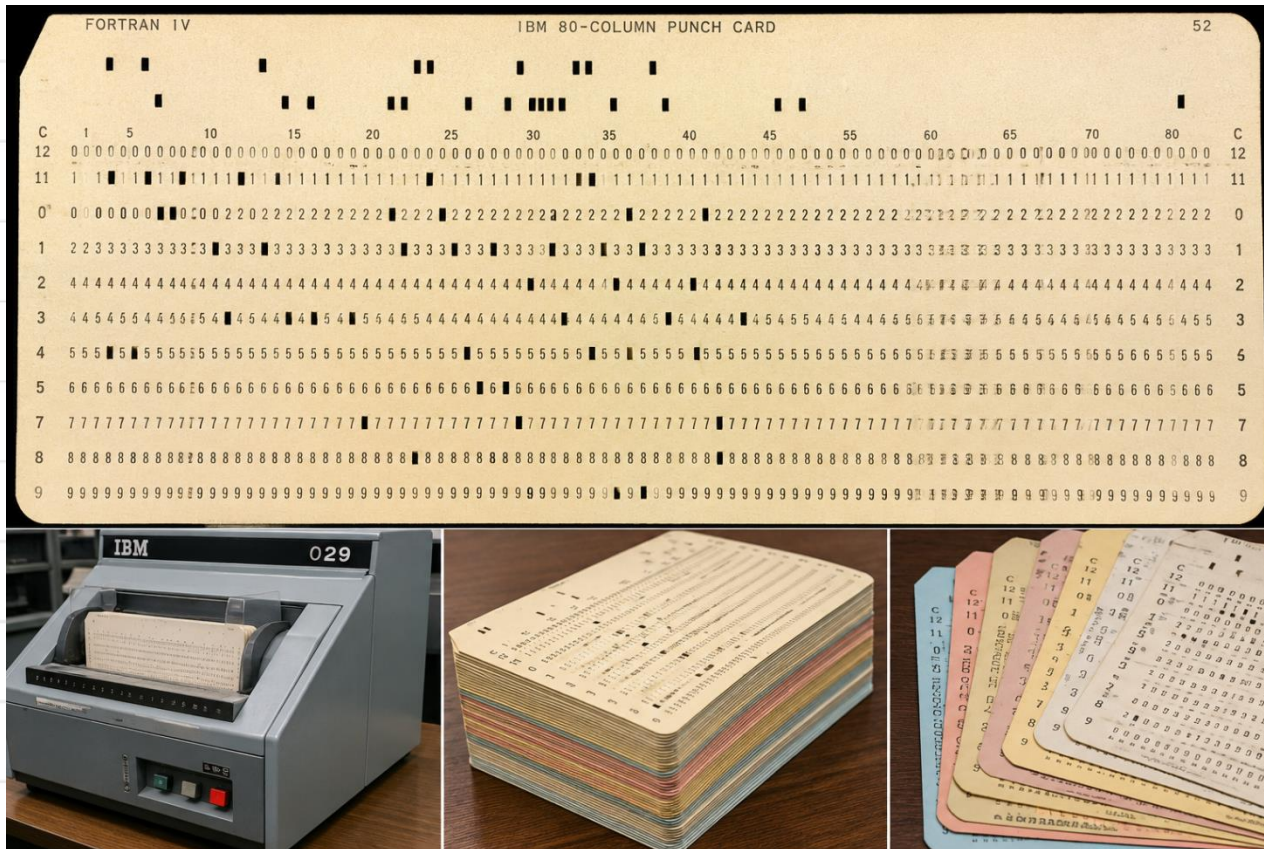
- Simple Batch Operating System
- Multiprogramming Operating System
- Multiprocessor Operating System
- Distributed Operating System
- Real-time Operating System

Simple Batch Operating Systems

- Firstly, user prepares his job using punch cards.
- Then, he submits the job to the computer operator.
- Operator collects the jobs from different users and sort the jobs into batches with similar needs.
- Then, operator submits the batches to the processor one by one.
- All the jobs of one batch are executed together.



Punch Cards



Simple Batch Operating Systems (contd.)

- **Advantages:**

- Processors of the batch systems know how long the job would be when it is in queue
- Multiple users can share the batch systems
- The idle time for the batch system is very less
- It is easy to manage large work repeatedly in batch systems

- **Disadvantages:**

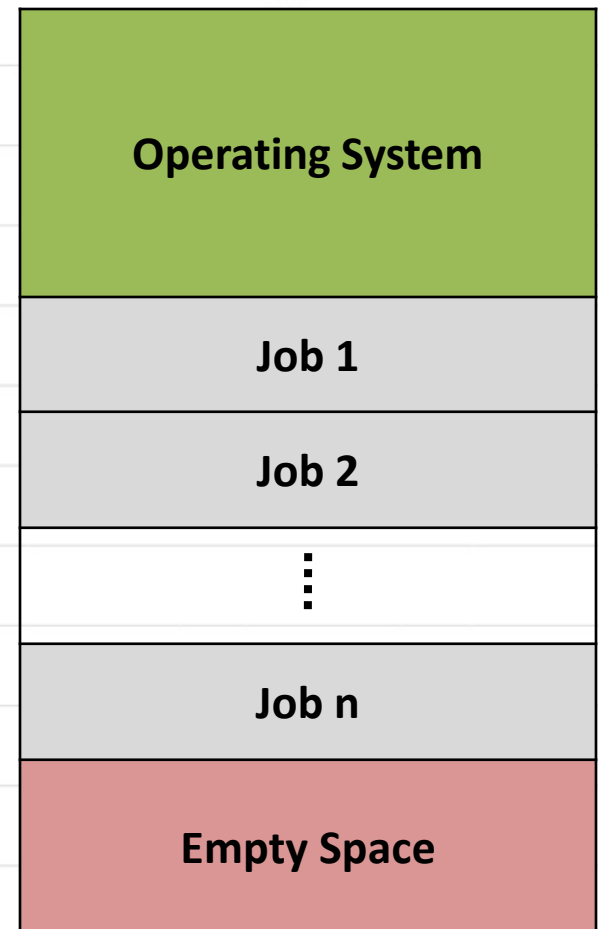
- The computer operators should be well known with batch systems
- Batch systems are hard to debug
- It is sometimes costly
- The other jobs will have to wait for an unknown time if any job fails

- **Examples:**

- Payroll System, Bank Statements, etc.

Multiprogramming Operating System

- Sharing the processor, when two or more programs reside in memory at the same time, is referred as **multiprogramming**.
- An OS does the following activities related to multiprogramming:
 - The operating system keeps several jobs in memory at a time.
 - This set of jobs is a subset of the jobs kept in the job pool.
 - The operating system picks and begins to execute one of the jobs in the memory.
 - Multiprogramming operating systems monitor the state of all active programs and system resources using memory management programs to ensures that the CPU is never idle, unless there are no jobs to process.



Multiprogramming Operating System (contd.)

- **Advantages:**

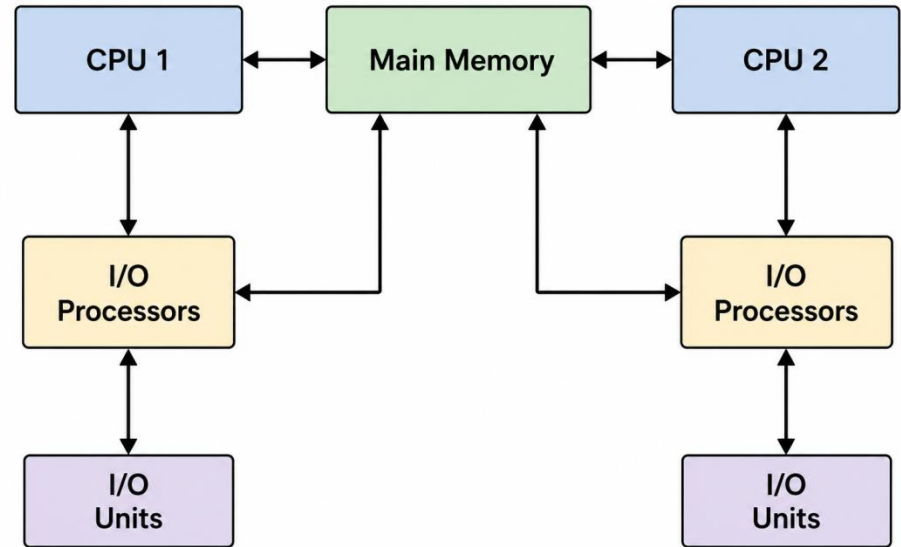
- High and efficient CPU utilization.
- User feels that many programs are allotted CPU almost simultaneously.

- **Disadvantages:**

- CPU scheduling is required.
- To accommodate many jobs in memory, memory management is required.

Multiprocessor Operating System

- A multiprocessor system consists of several processors that share a common physical memory.
- Multiprocessor system provides higher computing power and speed.
- In multiprocessor system all processors operate under single operating system.
- Multiplicity of the processors and how they do act together are transparent to the others.



Multiprocessor Operating System (contd.)

- **Advantages:**

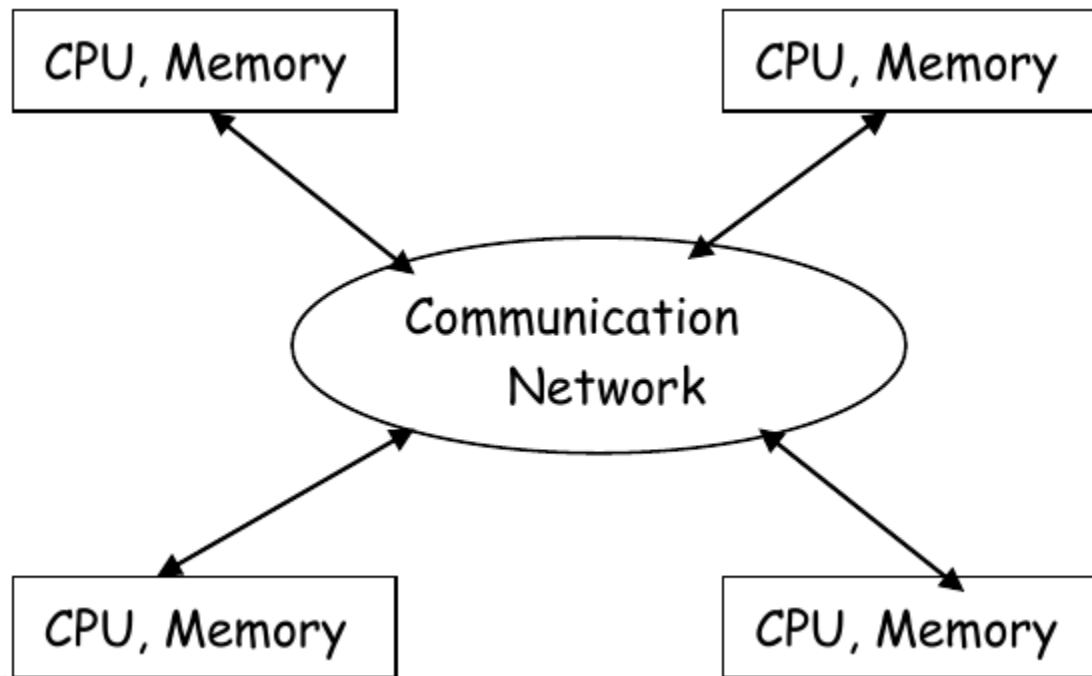
- Increased throughput (increased number of processed programs per second)
- Cost effective as it shares the same resources a single processor is using in a system.
- Increased reliability – Distributed workload results in increased reliability of the overall computer system.

- **Disadvantages:**

- Failure of even one of the processor negatively affects the overall speed of the operating system.
- These systems are expensive.
- More sophisticated operating systems are required to manage programs and data.
- The computer system requires a large main memory.

Distributed Operating System

- It is a collection of various autonomous but interconnected computers which communicate with each other using a *shared communication network*.
- All the systems are independent and possess their own memory unit and CPU.



Distributed Operating System (contd.)

- **Advantages:**

- Since all systems are independent, failure of any one will not affect the networking communication.
- Since resources are being shared, computation is highly fast and durable.
- Distributed systems are easily scalable as many systems can be easily added to the network.
- Data exchange process within the system in that network is very fast and reliable.

- **Disadvantages:**

- Since entire communication relies on a single network; failure of this network will stop the entire communication.
- Language used to establish distributed systems are not well defined yet.
- This is an expensive and not readily available system. Moreover, the underlying software is highly complex and not understood well yet.

Real-time Operating System

- Real-time systems are used when there are time requirements that are very strict like missile systems, air traffic control systems, robots, etc.
- The time interval required to process and respond to inputs is very small. This time interval is called *response time*.
- **Hard Real-Time Systems:**
 - These OSs are meant for applications where time constraints are very strict and even the shortest possible delay is not acceptable.
 - E.g., systems built for saving life like automatic parachutes or airbags.
- **Soft Real-Time Systems:**
 - These OSs are for applications where for time-constraint is less strict.
 - E.g., online gaming, email systems, weather monitoring systems

Real-time Operating System (contd.)

- **Advantages:**

- Maximum utilization of devices and system, thus more output from all the resources
- The time assigned for shifting tasks in these systems are very less.
- Focus on running applications and less importance to applications which are in the queue.
- Real-time operating system in the embedded system because of small size of programs.
- These types of systems are error-free.

- **Disadvantages:**

- Very few tasks run at the same time and their concentration is very less on few applications to avoid errors.
- Sometimes the system resources are not so good and they are expensive as well.
- The algorithms are very complex and difficult for the designer to write on.
- It needs specific device drivers and interrupts signals to respond earliest to interrupts.

Recap

1. **Consider a batch processing system where multiple jobs are grouped and executed sequentially without interaction. What is the main trade-off between throughput and responsiveness in such a system?**
 - a) High throughput, low responsiveness due to lack of interactivity
 - b) Low throughput, high responsiveness for each user
 - c) Low throughput, high I/O concurrency
 - d) High responsiveness due to dedicated CPU per job

2. **In multiprogramming systems, why is CPU scheduling a critical requirement?**
 - a) Because I/O devices operate faster than CPU
 - b) To allocate equal memory to all jobs
 - c) Because multiple jobs compete for the CPU and must be scheduled efficiently
 - d) To prevent memory fragmentation across multiple users

Recap

3. Which of the following limitations is *most likely* to affect a multiprocessor system as it scales up?
 - a) Unpredictable thermal shutdown
 - b) Linear increase in system throughput
 - c) Complexity in synchronization and shared memory access
 - d) Lack of multitasking capabilities in the kernel

4. Which of the following scenarios best justifies the use of a real-time operating system (RTOS)?
 - a) Batch processing of payroll at the end of the month
 - b) Web server handling client requests with caching
 - c) A drone's flight control system reacting to sensor data
 - d) Data analytics pipeline performing offline training of ML models

Recap

5. **In distributed operating systems, how is transparency (e.g., location, replication) typically achieved?**
 - a) By virtualizing the hardware across all nodes
 - b) By ensuring each node runs an identical OS kernel
 - c) Through middleware that abstracts the physical distribution of resources
 - d) By using batch scheduling to isolate remote processes

6. **Which is a fundamental challenge in designing distributed OS compared to centralized ones?**
 - a) Writing I/O drivers for legacy hardware
 - b) Enforcing real-time constraints on all tasks
 - c) Achieving fault tolerance and consistency across loosely coupled nodes
 - d) Ensuring all processes execute in kernel mode for efficiency